

REQUIREMENTS & DESIGN DOCUMENT — V1.0

EV ADAS System

Real-Time Electric Vehicle Dashboard

STM32F103C8T6 Blue Pill · PICSimLab · HC-SR04 Ultrasonic Sensors

Document Version 1.0 — Final	Date April 2026	Status Active
Platform STM32F103C8T6 (Blue Pill)	Simulator PICSimLab	Domain Automotive Embedded — ADAS + EV
Framework STM32 HAL (CubeIDE)	Sensors 3 × HC-SR04 Ultrasonic (Front, Left, Right)	

1 Introduction

1.1 Purpose

This document defines the functional and non-functional requirements, system architecture, hardware design, software design, and interface specifications for the EV ADAS System. The system consists of an STM32F103C8T6 (Blue Pill) microcontroller that monitors electric vehicle parameters in real time, implements ADAS collision and blind-spot warnings via HC-SR04 ultrasonic sensors, executes a deterministic vehicle state machine, and streams telemetry to a Python dashboard over UART.

1.2 Scope

Area	Description
Real-Time EV Sensing	Speed, SOC, motor torque, power, and range monitoring at 10 Hz
ADAS Safety	Forward collision warning, blind-spot detection, and parking assist via 3 HC-SR04 sensors
Vehicle State Machine	Deterministic PARKED → READY → DRIVING → REGEN → FAULT transitions
Telemetry Streaming	Structured binary packets over UART to Python dashboard at 10 Hz
Alarm Escalation	Priority-tiered alarm system (P0–P3) with LED, buzzer, and dashboard indication
Fault Management	Watchdog, fault isolation, and safe-state enforcement on detection

1.3 Definitions & Abbreviations

Term / Acronym	Definition
ADAS	Advanced Driver Assistance System
EV	Electric Vehicle
SOC	State of Charge (battery %)
TTC	Time To Collision (seconds)
HAL	Hardware Abstraction Layer (STM32)
PWM	Pulse Width Modulation
ISR	Interrupt Service Routine
DMA	Direct Memory Access
UART	Universal Asynchronous Receiver-Transmitter
HC-SR04	Ultrasonic distance sensor, 2–400 cm range
PICSimLab	Open-source PIC/ARM simulator used as the hardware simulation environment

2 Stakeholders & Use Cases

2.1 Stakeholders

Stakeholder	Role	Interest
Embedded Engineer	Implements firmware	Correct HAL driver usage, real-time timing, UART protocol
Test / QA Engineer	Validates system	Test case coverage, fault injection, alarm accuracy
Internship Supervisor	Reviews deliverables	Architecture quality, code modularity, documentation
Dashboard Operator	Monitors EV in simulation	Live speed, SOC, ADAS alerts, fault events
System Developer	Build, deploy, maintain	State machine correctness, sensor integration, watchdog behaviour

2.2 Use Cases

UC-01: Real-Time EV Dashboard Monitoring

An operator observes the Python dashboard showing live speed (km/h), SOC (%), motor torque (Nm), and estimated range. If any metric enters a warning zone, the corresponding panel highlights amber; a fault drives it red.

UC-02: Forward Collision Warning

The vehicle is driving at 72 km/h. The front HC-SR04 sensor reads 30 cm, giving a TTC of 0.015 s. The system immediately raises a CRITICAL alarm, activates rapid buzzer beep, flashes the red LED, and transitions to FAULT state, cutting motor PWM.

UC-03: Blind-Spot Detection

A vehicle enters the left blind spot when driving above 20 km/h. The left HC-SR04 reads 20 cm (<30 cm threshold). The ADVISORY alarm activates the yellow LED on PB9, a single beep sounds, and the dashboard bird-eye view highlights the left zone amber.

UC-04: Parking Assist

Vehicle speed drops below 10 km/h. Parking Assist mode activates and all three sensors provide a progressive proximity alert displayed on the dashboard bird-eye view with distance labels.

UC-05: Fault Recovery via UART Shell

After a motor overheat fault is triggered, an engineer issues the UART shell command 'fault clear'. The system transitions from FAULT back to PARKED state, clears all LEDs and alarms, and resumes normal operation.

3 Functional Requirements

3.1 EV Controller

ID	Requirement	Priority
FR-01	The system shall read accelerator and brake pedal ADC inputs at 100 Hz.	High

ID	Requirement	Priority
FR-02	The system shall compute vehicle speed using a physics-based inertia model.	High
FR-03	The system shall estimate SOC and deduct based on power draw over time.	High
FR-04	The system shall estimate remaining range from SOC and drive-mode efficiency.	Medium
FR-16	The system shall support ECO / NORMAL / SPORT drive modes with torque scaling.	Medium

3.2 ADAS Engine

ID	Requirement	Priority
FR-05	The system shall trigger all 3 HC-SR04 sensors every 100 ms in sequence.	High
FR-06	The system shall compute TTC from front sensor distance and current speed.	High
FR-07	The system shall issue collision WARNING when front distance < 50 cm or TTC < 3 s.	High
FR-08	The system shall issue collision CRITICAL when front distance < 20 cm or TTC < 1.5 s.	High
FR-09	The system shall detect left/right blind-spot vehicle when side distance < 30 cm.	High
FR-10	The system shall activate parking assist mode when speed < 10 km/h.	Medium

3.3 Fault & Safety

ID	Requirement	Priority
FR-11	The system shall transition to FAULT state on motor overheat (>90 °C) or SOC < 2%.	High
FR-12	The system shall cut PWM motor output immediately on FAULT state entry.	High
FR-17	The system shall activate buzzer tones mapped to alarm priority level.	Medium

3.4 UART & Telemetry

ID	Requirement	Priority
FR-13	The system shall stream EV metrics packet (0x01) every 100 ms over	High

ID	Requirement	Priority
	UART.	
FR-14	The system shall stream ADAS alert packet (0x02) every 100 ms over UART.	High
FR-15	The system shall respond to UART shell commands within one control loop cycle.	Medium

4 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Main EV control loop execution time < 5 ms
NFR-02	Performance	HC-SR04 ReadAll() worst-case time (3 sensors × 30 ms) < 100 ms
NFR-03	Performance	UART packet transmission latency < 10 ms
NFR-04	Real-Time	Fault state entry latency from detection < 1 loop cycle
NFR-06	Communication	UART baud rate 115200 bps
NFR-07	Sensing	ADC resolution 12-bit (4096 steps)
NFR-08	Sensing	Sensor polling interval 100 ms
NFR-09	Actuation	PWM motor drive frequency ~20 kHz
NFR-10	Accuracy	Distance measurement accuracy (HC-SR04 at 2–400 cm) ± 3 mm

5 System Architecture

5.1 High-Level Architecture

Tier	Component	Technology	Responsibility
Perception (Edge)	STM32 Blue Pill	STM32F103C8T6 + HAL	Read sensors, run state machine, drive motor PWM and alarms
Sensing	3× HC-SR04	Ultrasonic GPIO poll	Measure front, left, right obstacle distances
Transport	UART (DMA + IRQ)	USART1 @ 115200 bps	Binary telemetry stream and command shell to host PC
Application	Python Dashboard	matplotlib + pyserial	Live EV gauges, ADAS bird-eye view, alert display

5.2 Data Flow

- ADC inputs (accelerator, brake, SOC, motor temp) sampled at 100 Hz via TIM1 IRQ flag.
- EV module computes speed (inertia model), torque, power, SOC, and range each 10 ms cycle.
- TIM3 IRQ fires at 100 ms; HC-SR04 ReadAll() triggers and reads all 3 sensors sequentially.
- ADAS engine evaluates distances and TTC; raises alarm priority (P0–P3) and triggers buzzer/LED.
- Fault Manager evaluates fault flags; transitions to FAULT state and cuts PWM if required.
- UART TelemetryManager serialises EV + ADAS packets (0x01, 0x02) and transmits via DMA at 10 Hz.
- Python dashboard receives packets, parses binary frames, and refreshes gauges at 10 Hz.
- UART shell processes incoming commands (ring buffer + IRQ), dispatching to the relevant module.

5.3 Vehicle State Machine

State	Entry Condition	Exit Condition	Actions
PARKED	System power-on / reset	Accel pedal > 2%	All outputs off, UART heartbeat
READY	Accel pedal > 2%	Accel pedal > 5%	Motor armed, sensors active
DRIVING	Accel pedal > 5%	Brake > 10% or FAULT	Motor PWM, ADAS active, stream data
REGEN	Brake pedal > 10% while DRIVING	Brake < 5%	Negative torque PWM, SOC charging
FAULT	Overheat / low SOC / collision critical	Manual UART reset	Cut PWM, open contactors, alarm

5.4 Module Decomposition

Module	File	Responsibility
EV Controller	ev_control.c/.h	Speed model, SOC, torque, power, range, drive modes
ADAS Engine	adas.c/.h	TTC calc, collision/blind-spot/parking logic, alarm trigger
Ultrasonic Driver	ultrasonic.c/.h	HC-SR04 trigger-echo GPIO polling, distance conversion
Fault Manager	fault.c/.h	Fault flag evaluation, state transition, contactor control
UART Shell	uart_shell.c/.h	Command parser, ring buffer RX, packet TX via DMA
Buzzer Driver	buzzer.c/.h	TIM4 PWM tone generation, alarm priority mapping
Main Scheduler	main.c	Flag polling, ISR callbacks, watchdog refresh, init

6 Hardware Design

6.1 Node — Pin Assignment (Blue Pill)

Pin	Component	Direction	Notes
PA0	ADC1 CH0	Input	Accelerator pedal position (0–100%) via potentiometer
PA1	ADC1 CH1	Input	Brake pedal pressure (0–100%) via potentiometer
PA2	ADC1 CH2	Input	Battery SOC simulation via potentiometer
PA3	ADC1 CH3	Input	Motor temperature (NTC) via potentiometer
PA8	TIM1 CH1 PWM	Output	Motor drive signal (20 kHz)
PA9	USART1 TX	Output	Dashboard packet stream (DMA)
PA10	USART1 RX	Input	Command shell (IRQ ring buffer)
PC13	GPIO Output	Output	HC-SR04 Front TRIG
PB1	GPIO Input	Input	HC-SR04 Front ECHO
PB2	GPIO Output	Output	HC-SR04 Left TRIG
PB3	GPIO Input	Input	HC-SR04 Left ECHO
PB4	GPIO Output	Output	HC-SR04 Right TRIG
PB5	GPIO Input	Input	HC-SR04 Right ECHO
PB8	GPIO Output	Output	Red LED — Collision warning
PB9	GPIO Output	Output	Yellow LED — Left blind spot
PB10	GPIO Output	Output	Yellow LED — Right blind spot
PB11	GPIO Output	Output	Red LED — Fault indicator
PB0 (TIM3)	TIM3 CH1–3 PWM	Output	Buzzer tone generation

6.2 Component Specifications

Component	Model / Spec	Qty	Key Specification
Microcontroller	STM32F103C8T6 (Blue Pill)	1	72 MHz, 64 KB Flash, 20 KB SRAM, ST-Link V2
Ultrasonic Sensor	HC-SR04	3	2–400 cm range, ±3 mm accuracy, 5V supply
Passive Buzzer	PWM buzzer	1	Driven by TIM4 PWM, variable frequency tones
Status LEDs	Red / Yellow GPIO-	4	220 Ω series resistor; PB8–PB11

Component	Model / Spec	Qty	Key Specification
	driven		
Potentiometers (sim)	PICSimLab component	4	Simulate accel, brake, SOC, motor temp inputs
Signal Generator (sim)	PICSimLab component	1	Inject test waveforms
Oscilloscope (sim)	PICSimLab component	1	Observe motor PWM on PA8
UART Terminal (sim)	PICSimLab component	1	Dashboard packets + command shell
Programming interface	ST-Link V2	1	SWD programming and debug

6.3 HC-SR04 Sensor Zones

Sensor ID	TRIG	ECHO	Zone	Alert Threshold
SENSOR_FR ONT	PB0	PB1	Front (0–4 m)	WARNING < 50 cm or TTC < 3 s, CRITICAL < 20 cm or TTC < 1.5 s
SENSOR_LE FT	PB2	PB3	Left side	Blind spot < 30 cm (speed > 20 km/h)
SENSOR_RIG HT	PB4	PB5	Right side	Blind spot < 30 cm (speed > 20 km/h)

7 Software Design

7.1 Module Architecture

Module	Files	Responsibility
Configuration	config.h (implicit)	Pin assignments, thresholds, baud rate, timing constants
Sensor Manager	ultrasonic.c/.h	HC-SR04 trigger-echo polling, distance conversion, clamping
Actuator Manager	buzzer.c/.h	TIM4 PWM tone generation, LED GPIO control
EV Controller	ev_control.c/.h	Speed model, SOC integration, torque/power, drive modes
ADAS Engine	adas.c/.h	TTC, collision/blind-spot/parking logic, alarm priority
Fault Manager	fault.c/.h	Fault flag evaluation, state transitions, contactor control
UART Shell	uart_shell.c/.h	Ring buffer RX, command parser, binary packet

Module	Files	Responsibility
		TX via DMA
Main Scheduler	main.c	setup(), loop(), ISR flag polling, watchdog refresh

7.2 Telemetry Keys — EV Metrics (Packet 0x01)

Key	Type	Source	Description
speed	float32	EV model	Vehicle speed in km/h; physics-based inertia model output
soc	float32	EV model	State of Charge %; decrements with power draw, increments on regen
torque	float32	EV model	Motor torque Nm; scaled by drive mode (ECO 0.6×, SPORT 1.3×)
power_kw	float32	EV model	Instantaneous motor power kW; negative during regen braking
range_km	float32	EV model	Estimated remaining range km based on SOC and efficiency
motor_temp	float32	ADC / PA3	Motor temperature °C; FAULT triggered above 90 °C
drive_mode	uint8	Shell cmd	0=ECO, 1=NORMAL, 2=SPORT
uptime_ms	uint32	millis()	Milliseconds since last reset

7.3 Telemetry Keys — ADAS Alerts (Packet 0x02)

Key	Type	Source	Description
front_cm	float32	HC-SR04 Front	Front obstacle distance cm; clamped to [2, 400]
left_cm	float32	HC-SR04 Left	Left side distance cm
right_cm	float32	HC-SR04 Right	Right side distance cm
collision_lvl	uint8	ADAS engine	0=NONE, 1=WARNING, 2=CRITICAL
blindspot_l	uint8	ADAS engine	1 = left blind spot occupied
blindspot_r	uint8	ADAS engine	1 = right blind spot occupied
ttc_sec	float32	ADAS engine	Time To Collision seconds; front_cm / speed_cm_per_s

7.4 UART Shell Commands

Command	Example	Effect
mode <eco/normal/sport>	mode sport	Switch drive mode; reset torque scale
speed set <kmh>	speed set 80	Inject speed into EV model (test)
soc set <pct>	soc set 45	Override SOC value (test)
obstacle <cm>	obstacle 15	Inject front distance (test alarm)
blindspot <on/off>	blindspot on	Simulate side vehicle present
fault inject <type>	fault inject motor	Trigger motor overheat fault
fault clear	fault clear	Clear faults, return to PARKED
status	status	Print full system state over UART
stream <on/off>	stream on	Enable/disable UART packet stream
reset	reset	Software reset (triggers IWDG)

7.5 Auto-Fault Logic — Fault Manager

- Motor temp > 90 °C (FAULT_OT): Cut PWM, enter FAULT state.
- SOC < 2% (FAULT_SOC): Cut PWM, enter FAULT state.
- Collision CRITICAL (FAULT_COL): Cut PWM, enter FAULT state.
- Sensor timeout all 3 (FAULT_SEN): Warning log, degrade gracefully.
- UART comm timeout > 5 s (FAULT_COM): Warning log only.
- Watchdog not refreshed: IWDG hardware reset.

7.6 Alarm Hysteresis

Alarm de-assertion requires the triggering condition to be absent for at least 3 consecutive sensor readings (300 ms hysteresis) to prevent rapid alarm toggling on noisy sensor readings. This applies to all P1–P3 alarm conditions.

8 Dashboard & UI Design

8.1 EV Dashboard Overview

A Python GUI (matplotlib + pyserial) renders a Tesla-style dark instrument cluster. The dashboard provides a unified view of all EV and ADAS metrics refreshed at 10 Hz.

Panel	Location	Content	Update Rate
Speedometer	Top-left	Arc gauge, speed km/h, colour-coded green→amber→red	10 Hz
Battery SOC	Top-right	Horizontal bar, % label, estimated range km	10 Hz

Panel	Location	Content	Update Rate
ADAS Bird-Eye	Bottom-left	Top-down view: ego vehicle, obstacles, TTC, arcs	10 Hz
Metrics Panel	Bottom-right	Power kW, motor temp, torque Nm, drive mode badge	10 Hz

8.3 ADAS Bird-Eye View

Widget Type	Data Key	Configuration
Bird-Eye Canvas	front_cm, left_cm, right_cm	Top-down vehicle rectangle; obstacle arcs at measured distances
TTC Label	ttc_sec	Overlaid on front zone; colour changes with collision_lvl
LED Indicator	blindspot_l	ON=Amber / OFF=Grey Label: Left Blind Spot
LED Indicator	blindspot_r	ON=Amber / OFF=Grey Label: Right Blind Spot
LED Indicator	collision_lvl	NONE=Green / WARNING=Amber / CRITICAL=Red flash
Alarm Banner	collision_lvl	Full-width red banner with FAULT text on CRITICAL

8.4 Dashboard Colour Coding

Colour	Meaning
#00ff88 Green	Normal operating range — speed, SOC > 30%, no alerts
#ffaa00 Amber	Warning state — SOC 10–30%, collision warning, side vehicle
#ff3333 Red	Critical state — SOC < 10%, collision critical, fault
#4488ff Blue	Ego vehicle representation in bird-eye view
#ffffff White	Lane markings, distance labels

9 Communication Design

9.1 UART Packet Topics

Type	Name	Rate	Description
0x01	EV Metrics	10 Hz	speed, SOC, torque, power, range, motor_temp, drive_mode, uptime
0x02	ADAS Alerts	10 Hz	front_cm, left_cm, right_cm, collision_lvl, blindspot_l/r, ttc_sec

Type	Name	Rate	Description
0x03	Vehicle State	On change	state, drive_mode, fault_flags (3 × uint8)
0x04	ACK / Status	On request	uptime_ms (uint32) + version string

9.3 Communication Prerequisites

- UART baud rate: 115200 bps, 8N1.
- Python pyserial must open the correct COM/tty port connected to the Blue Pill's USART1.
- DMA must be enabled for USART1 TX in CubeMX to avoid blocking packet transmission.
- Ring buffer RX (128 bytes) must be initialised before the main loop begins.

10 Alarm & Rule Engine Design

10.1 Alarm Thresholds

Parameter	Warning	Critical	Action
Front distance	< 50 cm or TTC < 3 s	< 20 cm or TTC < 1.5 s	WARNING→CRITICAL alarm; FAULT + PWM cut on CRITICAL
Left/Right blind spot	< 30 cm (speed > 20 km/h)	N/A	ADVISORY alarm; Yellow LED + single beep
Motor temperature	> 70 °C advisory	> 90 °C	FAULT state entry; RED LED; cut PWM
Battery SOC	< 10%	< 2%	FAULT state entry on CRITICAL
Speed overspeed	> 120 km/h	N/A	Advisory alarm only; amber dashboard
Sensor timeout	Any sensor miss	N/A	Degrade gracefully; log FAULT_SEN warning
UART comm timeout	> 5 s	N/A	Warning log; dashboard shows last known values

10.2 Alarm Priority Levels

Level	Condition	Trigger	LED	Buzzer / Dashboard
P1 CRITICAL	Collision < 20 cm or TTC < 1.5 s	Immediate FAULT transition	Red rapid flash	Rapid beep; red flash + FAULT banner
P2	Collision < 50 cm	ADAS WARNING	Red steady	Double beep; orange

Level	Condition	Trigger	LED	Buzzer / Dashboard
WARNI NG	or TTC < 3.0 s	state		highlight
P3 ADVIS ORY	Blind spot / overspeed	ADAS advisory	Yellow steady	Single beep; yellow indicator
P0 NONE	All clear	Normal operation	Off	Silent; green normal

11.3 Acceptance Criteria

- Both EV and ADAS UART packets (0x01, 0x02) publish correctly within 10 seconds of power-on.
- Python dashboard gauges refresh within 150 ms of a sensor value change.
- FAULT state entry occurs within 1 main loop cycle (≤ 10 ms) of threshold breach.
- Motor PWM cuts to zero within 1 loop cycle of FAULT state entry.
- A P1 CRITICAL alarm is raised within 100 ms of front distance crossing the 20 cm threshold.
- System recovers from FAULT to PARKED correctly via 'fault clear' UART command.
- Watchdog resets the system correctly if the main loop stalls beyond 1 s.
- Free SRAM remains sufficient for ring buffer and packet buffers during normal operation.